

Table 1: Descriptive statistics of the 806 repositories using Cursor, collected at the time of data collection (April 2025).

	Mean	Min	25%	Median	75%	Max
Age (days)	1,002.3	283	361.5	521.5	1164.2	6,208
Stars	1,475.3	10	20.0	51.0	242.0	122,280
Forks	215.9	0	3.0	9.0	37.0	51,745
Contributors	19.2	0*	1.0	3.0	10.0	461
Commits	1,816.6	1	49.0	209.0	951.8	86,954
Issues	1,070.3	0	3.0	31.5	232.5	100,614
Pull Requests	719.6	0	1.0	18.0	161.8	72,015

* The GitHub API will return zero contributors for a repository if none of its commits can be mapped back to a GitHub user.

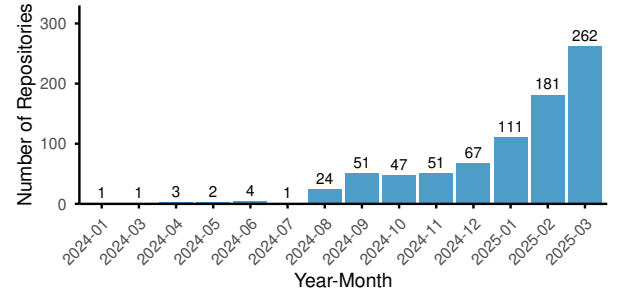
repository to direct AI behavior [7], leading to an adoption event proxy timestamp in the version control history and a scalable identification strategy based on these files. However, this identification strategy also has important limitations (Section 3.5), which we will address through robustness checks (Section 4.3).

3.1.2 Identifying GitHub Projects Adopting Cursor. We identify Cursor-adopting repositories and track adoption dates through configuration files in the git history. In the GitHub code search API [14], we query for repositories with `.cursorrules` files or `.cursor` folders. Since the API limits results to 1,000 per query, we implement an adaptive partitioning algorithm based on file sizes: For each query with size interval $[a, b]$, we create two queries $[a, (a+b)/2]$ and $[(a+b)/2, b]$ until results fall below 1,000. This discovered 23,308 Cursor files across 3,306 non-fork repositories as of March 2025.

To filter non-software, educational, toy, and spam repositories [60, 68], we follow prior work [59, 62, 103] by requiring at least 10 stars at collection time—a threshold achieving 97% precision in identifying engineered projects [84]. This yields 806 repositories with adoption dates between January 2024 and March 2025 that are still available on GitHub at the time of data analysis (August 2025).

As expected, the dataset is highly skewed across many repository-level metrics (Table 1), and adoption time is staggered, with adoptions growing over time (Figure 2). These dataset characteristics motivate us to adopt a DiD design with staggered adoption and a matched control group, as we will discuss in the remainder of this section. While we did not filter based on activity levels here, activity-based subsets will be used as part of our robustness checks. The top five primary programming languages in our dataset are: TypeScript (366 repositories), Python (118 repositories), JavaScript (60 repositories), Go (36 repositories), and Rust (24 repositories).

3.1.3 Building a Control Group via Propensity Score Matching. A staggered DiD design usually requires a “never-treated” group to serve as comparison units, which, in our case, means repositories that never adopted Cursor (i.e., *never-adopters*). This comparison allows for a causal interpretation under the parallel trend assumption: Repositories that adopted Cursor should, on average, evolve similarly in the absence of Cursor adoption compared to the control group. However, this assumption is unlikely to hold if there are systematic differences between Cursor-adopting repositories and the control group. Repositories adopting Cursor may be more active,

**Figure 2: The Cursor adoption time of the 806 repositories in our study, which all have ≥ 10 stars and Cursor configuration files at the time of data collection (April 2025).**

more rapidly growing, and have larger communities compared to random never-adopter repositories, leading to biased estimates.

To address these potential confounders and build a comparable quasi-experimental control group, we use *propensity score matching* [26, 30]. The high-level idea is to estimate propensity scores (i.e., the probability of adoption conditional on pre-adoption covariates) for each Cursor-adopting repository and a large population of other GitHub repositories, and retain only never-adopter repositories with propensity scores similar to those of the control group. Specifically, we define this population as all GitHub repositories with ≥ 10 stars (matching our inclusion threshold for Cursor adopters). For each month with major Cursor adoptions (August 2024–March 2025), we collect monthly time series from GHArchive [1] for all repositories in the population: age, active users, stars, forks, releases, pull requests, issues, comments, and total events.

Rather than static snapshots, we fit propensity score models to capture *dynamics*: repositories experiencing rapid growth or changing patterns may be more likely to adopt new tools. Let T_{it} denote repository age and X_{it} denote remaining covariates for repository i at month t . We estimate propensity scores $P(\text{treat}|t, T_i, X_i)$ (i.e., the probability of getting treated given t and repository-level covariates T_i, X_i) via the following logistic regression:

$$\log \frac{P(\text{treat}|\dots)}{1 - P(\text{treat}|\dots)} = \alpha + \beta T_{i,t-1} + \sum_{j=1}^6 \Gamma_j X_{i,t-j} + \Theta \sum_{j=7}^{\infty} X_{i,t-j} + \epsilon_i \quad (1)$$

where α , β , Γ_j and Θ are regression parameters. This effectively captures: (1) *repository maturity* ($T_{i,t-1}$), as older repositories may have different adoption patterns, (2) *recent dynamics* ($\sum_{j=1}^6 \Gamma_j X_{i,t-j}$), i.e., month-by-month evolution over six months captures trends and growth, and (3) *historical baselines* ($\Theta \sum_{j=7}^{\infty} X_{i,t-j}$), i.e., cumulative history providing context on overall project scale and activity. By including lags, we ensure that propensity scores reflect both activity level and trajectory—two repositories with identical July 2024 pull requests may differ if one is growing while the other declines; such longitudinal characteristics may correlate with both Cursor adoption and long-term velocity and quality outcomes.

Since candidate repositories outnumber Cursor adopters by orders of magnitude, we sample at most 10,000 candidates per Cursor adoption month to avoid extreme imbalance and improve fit. This yields AUC values ranging from 0.83 to 0.91, indicating high